Practical NO.05

Name - Sakshi Vijaykumar Muthiyan

Roll No. -

Title - Implement Page Rank Algorithm. (Use python or beautiful soup for implementation).

In [3]:
```
import networkx as nx

def pagerank(G, alpha=0.85, personalization=None,
             max_iter=100, tol=1.0e-6, nstart=None, weight='weight',
             dangling=None):
    """Return the PageRank of the nodes in the graph."""
    if len(G) == 0:
        return {}

    if not G.is_directed():
        D = G.to_directed()
    else:
        D = G

    # Create a copy in (right) stochastic form
    W = nx.stochastic_graph(D, weight=weight)
    N = W.number_of_nodes()

    # Choose fixed starting vector if not given
    if nstart is None:
        x = dict.fromkeys(W, 1.0 / N)
    else:
        # Normalized nstart vector
        s = float(sum(nstart.values()))
        x = dict((k, v / s) for k, v in nstart.items())

    if personalization is None:
        # Assign uniform personalization vector if not given
        p = dict.fromkeys(W, 1.0 / N)
    else:
        missing = set(G) - set(personalization)
        if missing:
            raise nx.NetworkXError('Personalization dictionary '
                                   'must have a value for every node. '
                                   'Missing nodes %s' % missing)
        s = float(sum(personalization.values()))
        p = dict((k, v / s) for k, v in personalization.items())

    if dangling is None:
        # Use personalization vector if dangling vector not specified
        dangling_weights = p
    else:
        missing = set(G) - set(dangling)
        if missing:
            raise nx.NetworkXError('Dangling node dictionary '
```

```
                                    'must have a value for every node. '
                                    'Missing nodes %s' % missing)
        s = float(sum(dangling.values()))
        dangling_weights = dict((k, v/s) for k, v in dangling.items())

    dangling_nodes = [n for n in W if W.out_degree(n, weight=weight) == 0

    # power iteration: make up to max_iter iterations
    for _ in range(max_iter):
        xlast = x
        x = dict.fromkeys(xlast.keys(), 0)
        danglesum = alpha * sum(xlast[n] for n in dangling_nodes)
        for n in x:
            # this matrix multiply looks odd because it is
            # doing a left multiply x^T=xlast^T*W
            for nbr in W[n]:
                x[nbr] += alpha * xlast[n] * W[n][nbr][weight]
            x[n] += danglesum * dangling_weights[n] + (1.0 - alpha) * p[n

        # check convergence, l1 norm
        err = sum([abs(x[n] - xlast[n]) for n in x])
        if err < N * tol:
            return x

    raise nx.NetworkXError('pagerank: power iteration failed to converge
                           'in %d iterations.' % max_iter)
```

In [2]:

```
import networkx as nx

G = nx.barabasi_albert_graph(60, 41)
pr = nx.pagerank(G, 0.4)
print(pr)
```

```
{0: 0.028119443152935422, 1: 0.012960547281886434, 2: 0.01217710032972261
2, 3: 0.013371992112692096, 4: 0.011791580162602733, 5: 0.012360480044742
73, 6: 0.013181157827997309, 7: 0.013151782261451871, 8: 0.01198834423954
4728, 9: 0.01337575092821415, 10: 0.012964230607146599, 11: 0.01275706661
2273275, 12: 0.012965815074221664, 13: 0.013777469010187544, 14: 0.012748
854055635878, 15: 0.012771458274638792, 16: 0.013369938928806667, 17: 0.0
13365366152429285, 18: 0.013158807044841264, 19: 0.01356489174657445, 20:
0.013358043755050036, 21: 0.012970858622155362, 22: 0.01216668274326727,
23: 0.012356640308820848, 24: 0.012948770680031613, 25: 0.013158215207713
903, 26: 0.012767047080823254, 27: 0.01297874688680538, 28: 0.01316113653
640506, 29: 0.013159625465532726, 30: 0.012568849510706085, 31: 0.0127569
78350777114, 32: 0.01315768593465499, 33: 0.012978573637017942, 34: 0.013
163286404957933, 35: 0.012769237375660162, 36: 0.012762580289549972, 37:
0.012986005122235668, 38: 0.013364121579658387, 39: 0.013155530594496426,
40: 0.01295012264341859, 41: 0.013169191147725561, 42: 0.0277069950910672
14, 43: 0.027252589720312484, 44: 0.0267692009328675, 45: 0.0264754558672
59606, 46: 0.02596104536351248, 47: 0.025789235717569865, 48: 0.025408485
700794056, 49: 0.025179612485787456, 50: 0.024545649564784933, 51: 0.0245
9359028200145, 52: 0.02380948079473604, 53: 0.023329816606430034, 54: 0.0
2314525533130802, 55: 0.023203822153037353, 56: 0.02226814883127312, 57:
0.02232001038873724, 58: 0.02190158178149374, 59: 0.021610017661017653}
```